

Trabajo Práctico Final Inteligencia Artificial

Agustín Fernández Bergé y Ramiro Gatto

26 de julio de 2025

1. Introducción

La idea principal del proyecto es usar InceptionV3 para poder clasificar los distintos “sellos” (gestos con las manos) del anime de Naruto. Para hacer esto hicimos transfer learning sobre el modelo de InceptionV3 más un finetuning de las ultimas capas del modelo.

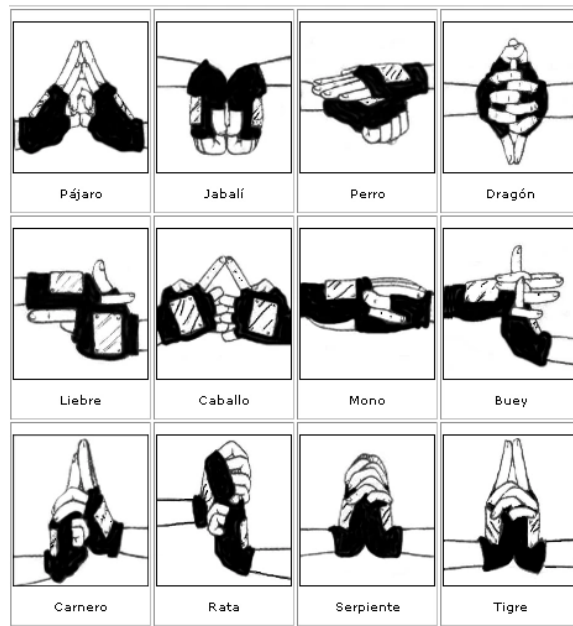


Figura 1: Sellos Basicos

Para el proyecto se utilizaron 3 dataset de proyectos similares (primero empezamos con uno y lo fuimos ampliando), más una cuantas imágenes que agregamos para que este sea un poco más variado.

2. Primer acercamiento

Como experiencia inicial quisimos probar como clasificaba InceptionV3 usando los pesos de Imagenet las imagenes del dataset, para ver en que categorias clasificaba cada sello. Nos dimos cuenta que en algunos sellos una clase se activaba mucho más que las otras, mientras que para otros sellos 3 o 5 clases eran las que mas se activaban respecto a otras. Aqui hay un ejemplo para varios sellos:

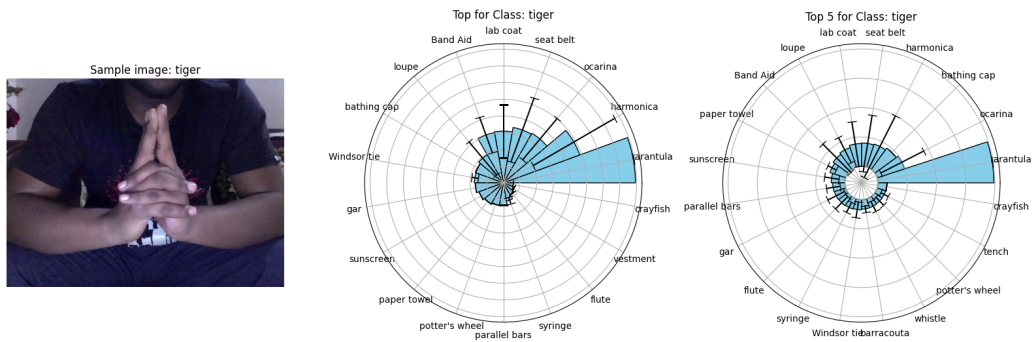


Figura 2: Clasificación de InceptionV3 para el sello del tigre

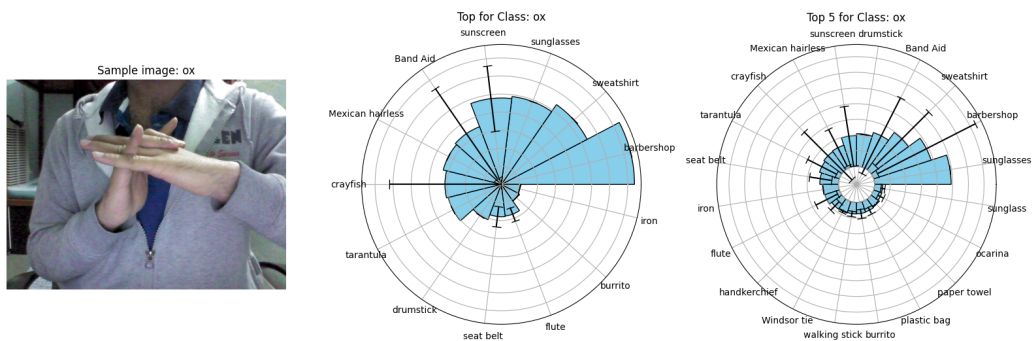


Figura 3: Clasificación de InceptionV3 para el sello del buey

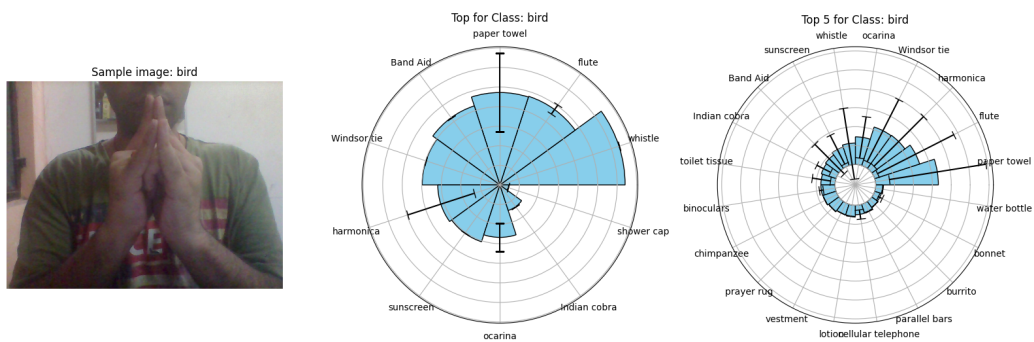


Figura 4: Clasificación de InceptionV3 para el sello del pajar

Por lo que si queriamos que inception clasificara los sellos, ibamos a necesitar hacer un refinamiento del modelo.

3. Primeros pasos

Lo primero que hicimos fue leer el materia subido en la pagina de Keras sobre transfer learning y fine-tuning [1]. Los pasos de dicho tutorial se pueden resumir como siguen:

1. Cargar los datasets de entrenamiento, validación y test.
2. Aplicar el preprocesamiento correspondiente para que el modelo pueda trabajar con las imágenes. En el caso de InceptionV3, el preprocesamiento consiste en redimensionar las imagenes a un tamaño común y normalizar los valores de los píxeles a valores del conjunto $[-1,1]$.
3. Aplicar data augmentation a los datasets de entrenamiento y validación(este paso inicialmente lo saltamos).
4. Construir las capas del modelo, dejando la capa de InceptionV3 congelada (es decir, sin entrenar) y agregando una capa densa al final para la clasificación.
5. Compilar el modelo, esto es, definir la función de loss, el optimizador y las métricas a utilizar.
6. Entrenar el modelo una cantidad determinada de epocas.
7. Descongelar el modelo de InceptionV3 y realizar un nuevo entrenamiento, esta vez con un learning rate más bajo.
8. Finalmente, evaluar el modelo en los datasets de entrenamiento, validación y test para evaluar rendimiento.

Modificamos un poco las sugerencias del tutorial y agregamos un par de cosas más. principalmente porque el tutorial esta orientado a clasificar imagenes en 2 clases y nosotros queríamos clasificar en 12 clases. Para esto hicimos lo siguiente:

- En vez de usar la función de loss Binary Crossentropy, usamos Categorical Crossentropy. Estas entropías no son las mismas que se usan en los arboles de decisión, basicamente la Binary Crossentropy se usa para clasificar en 2 clases mientras que la Categorical Crossentropy es una generalización de la Binary Crossentropy para clasificar en más de 2 clases.
- En la capa densa del final del modelo, usamos 12 neuronas (una por cada clase) en lugar de 2 y usamos como función de activación Softmax. Softmax es una generalización de la función Sigmoide y es utilizada como función de activación al clasificar en más de 2 clases.

- Cambiamos la metrica del modelo, en cada epoca se le puede indicar al modelo que calcule una serie de metricas para que las muestre al usuario. Nosotros le indicamos que muestre el accuracy de los conjuntos de entrenamiento y validación en lugar de la binary accuracy.
- Agregamos 3 callbacks para modificar el proceso de entrenamiento:
 - EarlyStopping: Si el accuracy del conjunto de validación no mejora en un número determinado de épocas, se detiene el entrenamiento y se restauran los pesos que obtuvieron el mejor accuracy.
 - ModelCheckpoint: Si durante el entrenamiento se obtiene una mejora en el accuracy del conjunto de validación, se guardan los pesos del modelo en un archivo en disco.
 - ReduceLROnPlateau: Si el accuracy del conjunto de validación no mejora en un número determinado de epocas (menor a las configuradas en EarlyStopping), se reduce el learning rate en un factor de 0.1.

3.1. El primer modelo

El primer modelo que hicimos tiene la siguiente estructura:

Layer (type)	Output Shape	Param #	Trainable
rescaling (Rescaling)	(None, 150, 150, 3)	0	-
inception_v3 (Functional)	(None, 3, 3, 2048)	21,802,784	N
global_average_pooling2d (GlobalAveragePooling2D)	(None, 2048)	0	-
dropout (Dropout)	(None, 2048)	0	-
dense (Dense)	(None, 13)	26,637	Y

Cuadro 1: Estructura del primer modelo

Reescaling Esta capa se encarga de normalizar los valores de los pixeles de las imagenes a $[-1, 1]$. Esto es necesario para que InceptionV3 pueda trabajar con las imagenes,

InceptionV3 La capa base del modelo, inceptionv3 esta cargado con los pesos de imagenet y se encuentra congelado, es decir, no se entrena durante el entrenamiento inicial.

GlobalAveragePooling2D Esta capa se encarga de reducir la dimensionalidad de la salida de InceptionV3.

Dropout Esta capa se encarga de apagar, de manera aleatoria, un porcentaje de las neuronas de la capa anterior para evitar el overfitting.

Dense Esta es la capa de salida del modelo, es una capa donde todas las neuronas de la capa anterior estan conectadas con 12 neuronas de salida, las cuales tienen una función de activación Sotmax.

El primer entrenamiento se realiza con la siguiente configuración:

```
model.compile(  
    optimizer=keras.optimizers.Adam(learning_rate=init_learning_rate),  
    loss='categorical_crossentropy',  
    metrics=['accuracy'],  
)
```

Optimizer=Adam Es el algoritmo que se usa para actualizar los pesos del modelo. En clase y en el tp de redes neuronales nosotros usabamos el optimizador SGD(Stochastic Gradient Descent), Adam presenta muchas mejoras sobre SGD(como learning rate y momentum adaptativo) y es bastante utilizado al entrenar redes neuronales. Con el optimizador tambien configuramos el learning rate inicial, tipicamente probabamos los valores 0.1, 0.01 y 0.001.

loss La función de perdida que la red neuronal intenta minimizar, como dijimos antes, usamos Categorical Crossentropy.

metrics Las metricas que calcula el modelo en cada epoca, en este caso usamos accuracy.

Cuando hacemos fine tunning del modelo, descongelamos una serie de capas de InceptionV3 y entrenamos usando un learning rate más bajo, al inicio usabamos un learning rate de $1e-5$.

3.2. Primeros problemas

.Llegamos a un modelo que posee el siguiente accuracy:

- Accuracy Train: Superior al 80 %
- Accuracy Validation: En torno al 50 %
- Accuracy Test: En torno al 20 %

Donde claramente se ve que ocurre (muy notablemente) overfitting. Ademas, cuando lo queríamos probar para ver si al menos clasificaba bien los ejemplos del train este los clasificaba casi siempre mal.

3.3. Primera Solución

Entonces como primer medida pensamos que el modelo podría clasificar mal las imágenes debido a que estas eran muy poco “claras”, debido a que eran de poca resolución y las manos eran un poco “particulares”. Sumándole el hecho de que hay algunos “sellos” muy parecidos (ejemplo, el sello del carnero y el sello del tigre), llegamos a una primera conclusión de que estos factores hacían que el modelo no aprendiese bien.

Entonces como primera solución que aplicamos fue ampliar el dataset, con otros dataset, con manos diferentes al que teníamos, más el agregado de imágenes que sacamos de este canal de youtube [2] el cual hace lo gestos con diferentes atuendos, dándonos el agregado de variedad que necesitábamos.

4. Continúan los problemas

4.1. Continua el problema

A pesar de haber agregado mas ejemplos y que haya mas variedad seguíamos teniendo overfitting. Era mucho menor teniendo los siguiente resultados para el train y test:

- Accuracy Train: En torno al 90
- Accuracy Test: En torno al 60

Además seguíamos teniendo el problema de que al probar con ejemplos del train, que deberían dar casi siempre bien por el alto accuracy, las clasificaba mal.

4.2. Segunda solución

Pensamos y aplicamos muchas soluciones, esta por separado no fueron muy útiles (mejoraban ligeramente) pero fue cuando las combinamos que se noto un poco mas la mejora; estas fueron:

- Como primer medida pensamos que el learning rate (α) no era demasiado chico (al crear el modelo y en el fine-tuning). Entonces lo redujimos en ambos casos, haciendo que el del fine-tuning sea menor y que se pueda achicar si es necesario al momento de entrenar.

- La subsecuente medida que se nos ocurrió fue cambiar el fine-tuning. Ahora en vez de congelar y descongelar la ultima capa, lo hacemos con mas (probamos primero con 50 y luego con 100, quedándonos con las 100)
- También, pensamos que los ejemplos actuales podían no ser suficientes, así que agregamos data augmentation para mitigar esto (ya no encontramos mas datasets)

5. Un nuevo problema

5.1. Nada es lo que parece

Con los cambios que explicamos en la sección anterior hubo una mejora en cuanto a los resultados. Ya el overfitting era mucho menos, la diferencia entre los accuracy del train y test era del solo del 10 % en casi todos los casos.

Pero seguíamos teniendo el problema de que clasificaba prácticamente siempre todas las imágenes. En este punto no entendíamos bien los que pasaba al hacer el test/train saca un accuracy alto (en ambos caso era mayor de 70 %) pero al probarlo nosotros siempre calificaba mal (con estos numero al probar 5 imágenes random debería clasificar al menos 3 bien pero nunca pasaba esto, casi siempre clasificaba mal las 5).

5.2. Busqueda del problema

En un inicio lo que pensamos es que el modelo confundía “sellos” que serian parecidos (como puede ser “rat”, “ram”, “tiger” o “snake”, “boar”).

Asi que lo que hicimos fue rehacer el modelo pero juntando las clases parecidas. De esta forma podríamos saber si en efecto las confunde o el problema es por otra coso.

Para nuestra desgracia el problema no era por ahi, al juntar las clases parecidas el modelo anda mucho mejor. Es decir con valores altos de los accuracies y no fallando al probarlo con ejemplos del dataset y con ejemplos por fuera de este.

Una solución que nos dieron fue ver las permutaciones de los 12 “sellos” en grupos de 3, ver cuales se podían juntar y por cada conjunto crear un modelo y a esto sumarle un modelo para poder ver a que grupo pertenecia. Asi el ultimo modelo me dice a que modelo debo mandar la imagen para que el siguiente modelo lo clasifique (esto fue lo que entendimos). Tratamos de ver si esto era posible, sacando las cuneta tendríamos que probar permutaciones de 3 clases de las 12 posibles. Si cada modelo tarda 30 minutos en hacerce tardariamos casi 40000 minutos o mas de 1500 días para obtener

todos los posibles modelos (algo que es casi imposible de lograr en esta vida), así que tratamos de buscar otra solución.

5.3. Solución que aplicamos

Viendo que la solución anterior era prácticamente imposible de realizar decidimos ver que otro parámetro podríamos modificar para que el problema desaparezca.

Entonces realizamos los siguientes cambios:

- Cuando realizábamos el *model.compile* utilizábamos *CategoricalCrossentropy()* para el *loss*, ahora realizamos *CategoricalFocalCrossentropy()*. Debido a que este último (COMPLETAR).
- Una cosa que sacamos fue el dropout al momento de crear el modelo. Si bien la idea del dropout es perturbar el modelo en cada iteración de entrenamiento para que pueda aprender de otras formas los conceptos, vimos que en nuestro caso es contraproducente porque ... (COMPLETAR)
- También agregamos la función de que corte el entrenamiento luego de cierto tiempo. Si bien usamos el *StopErlay* para que corte ante si no hay una mejora, a veces esta mejora era muy ínfima y hacía que el modelo tarde mucho. Al cortarlo antes nos ahorramos tener que esperar de más por una mejora del menos del 1 % en los accuracy.
Esto fue más que nada para poder probar un poco más de veces y evitar que la computadora se muera en el proceso.

Luego de realizar estos cambios el modelo tenía un accuracy de train mayor al 90 % y un accuracy de train mayor al 80 %, y ahora al probarlo clasificaba bien los ejemplos cuando los probábamos nosotros.

Referencias

- [1] fchollet. Transfer learning and fine-tuning. https://keras.io/guides/transfer_learning.
- [2] K.E.N-DIGIT / Hokage. Canal de youtube sobre los sellos. <https://www.youtube.com/@degidoga>.